Chapter 2
Supervised Learning

Part 2

Decision Trees and Ensemble Methods

Course Outline

Linear Models and Regularization	• Linear Regression • Logistic Regression • Regularization Techniques
Decision Trees and Ensemble Methods	• **Decision Trees** • **Bagging: Random Forests** • **Boosting** • **Stacking and Blending**
SVM and Advanced Optimization	• Introduction to Support Vector Machines (SVM) • Kernel trick: linear, polynomial, RBF, sigmoid • Optimization techniques: SGD, Adam

Decision Trees and Ensemble Methods Introduction

- Linear models (linear and logistic regression) are
  - simple, interpretable, and fast,
- but they rest on a **strong linearity assumption** between the explanatory variables and the output.
- In reality, the phenomena we observe are often **highly non-linear** and interactive.

Decision Trees and Ensemble Methods Introduction

- Several **limitations** have been encountered, such as:
  - Difficulty in modelling **complex interactions** between variables.
  - Sensitivity to **extreme values** and **noisy data**.
  - The need for **intensive feature engineering** in order to achieve good performance.

How can we build a model capable of learning **interactions between variables automatically** and of **handling non-linear relationships**, while remaining interpretable and performant?

Decision Trees and Ensemble Methods Introduction

- Linear models often require a **manual transformation** of the variables.
- **Decision trees**, by contrast, **learn these complex relationships directly** by partitioning the data space into homogeneous regions.

Decision Trees and Ensemble Methods Introduction

- **Illustrative example:**
- consider a classification problem whose goal is to determine whether a customer will purchase a product:
  - If age > 30 AND income > 50,000 DZD → Probably purchase
  - If age ≤ 30 AND time_on_site > 10 min → Probably purchase
  - Otherwise → No purchase
- This cascading decision logic can be represented naturally by a tree.

Decision Trees and Ensemble Methods Introduction

- **Decision trees** offer an intuitive and powerful approach:
  - They **recursively partition** the data space according to thresholds selected automatically.
  - Each **internal node** corresponds to a **decision question** (e.g., “is income > 30,000?”).
  - Each **leaf** corresponds to a **final prediction** (a class or a value).

- **Decision trees** therefore make it possible to **model non-linear and conditional relationships** while remaining **interpretable**.

Decision Trees and Ensemble Methods
Decision Trees

- **Definition**
- A **decision tree** is a **supervised learning model** used for both **classification** (Classification Tree) and **regression** (Regression Tree).
- It predicts the value of a target (categorical or continuous) by **recursively partitioning** the space of explanatory variables X according to simple **decision rules** of the form:

  "If $X_i < t$, then go left; otherwise, go right."

- Each **leaf** corresponds to a **predicted value** (a class or a mean)
- and each **internal node** corresponds to a **question about one variable**.

Decision Trees and Ensemble Methods
Decision Trees

- **Example** 1: predicting whether a student will pass an exam.
  Variables:
  - X_1= study hours
  - X_2= number of absences
  - X_3= continuous-assessment average
- A decision tree can automatically construct the following rules:

If (study hours > 15)
    If (absences < 3) → Pass
    Otherwise → Fail
Otherwise
    If (grade average > 10) → Pass
    Otherwise → Fail



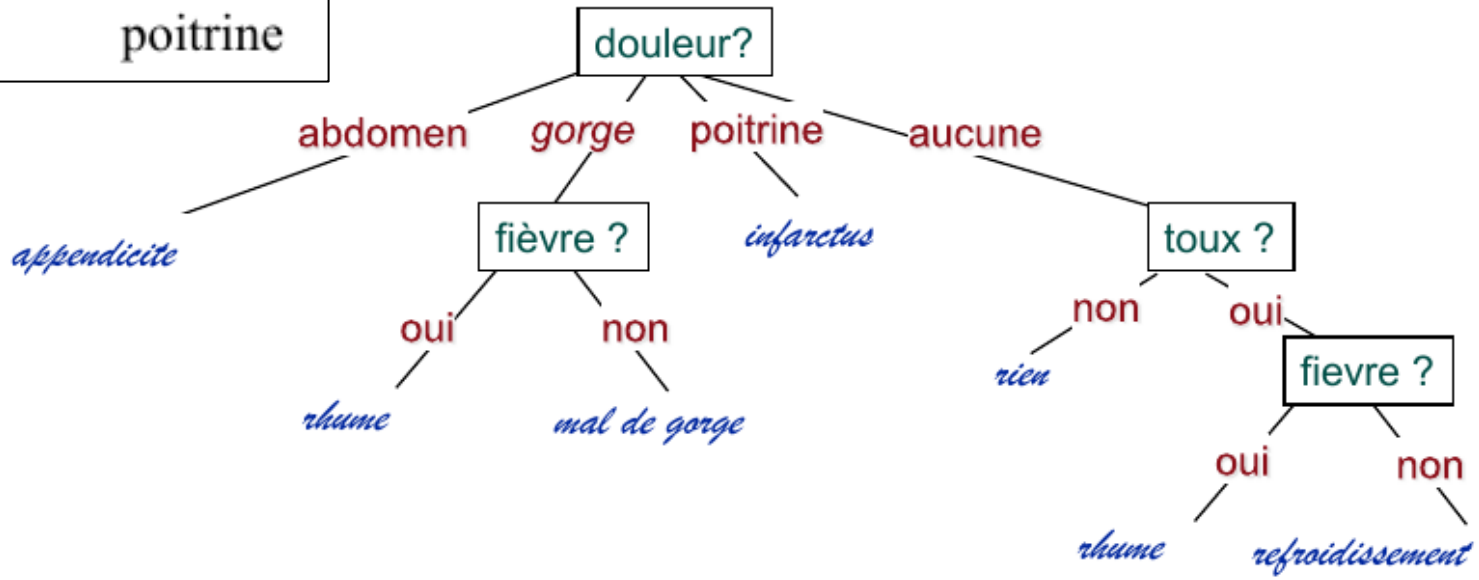
These rules are **learned from the data** (they are not imposed by a human).

Decision Trees and Ensemble Methods
Decision Trees

- **Example** 2:
- A set of individuals described by several categorical variables (Cough, Fever, Weight, Pain).

Toux	Fièvre	Poids	Douleur
non	oui	normal	gorge
non	oui	normal	abdomen
oui	oui	maigre	aucune
oui	non	obese	poitrine

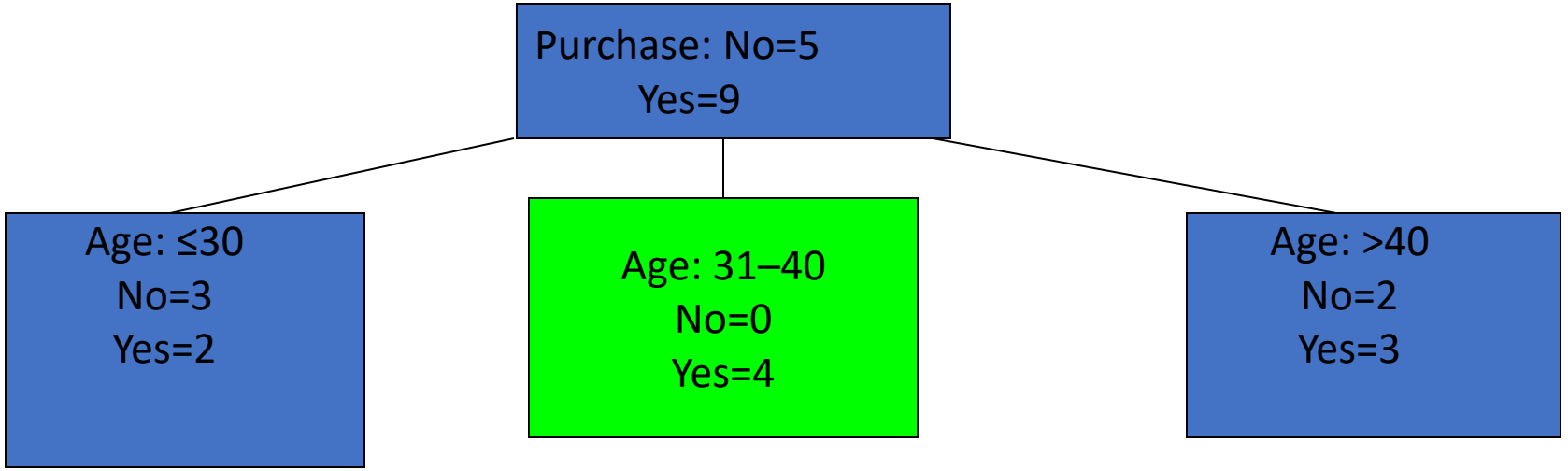
- In the resulting decision tree obtained, the target variable is found in the leaves.



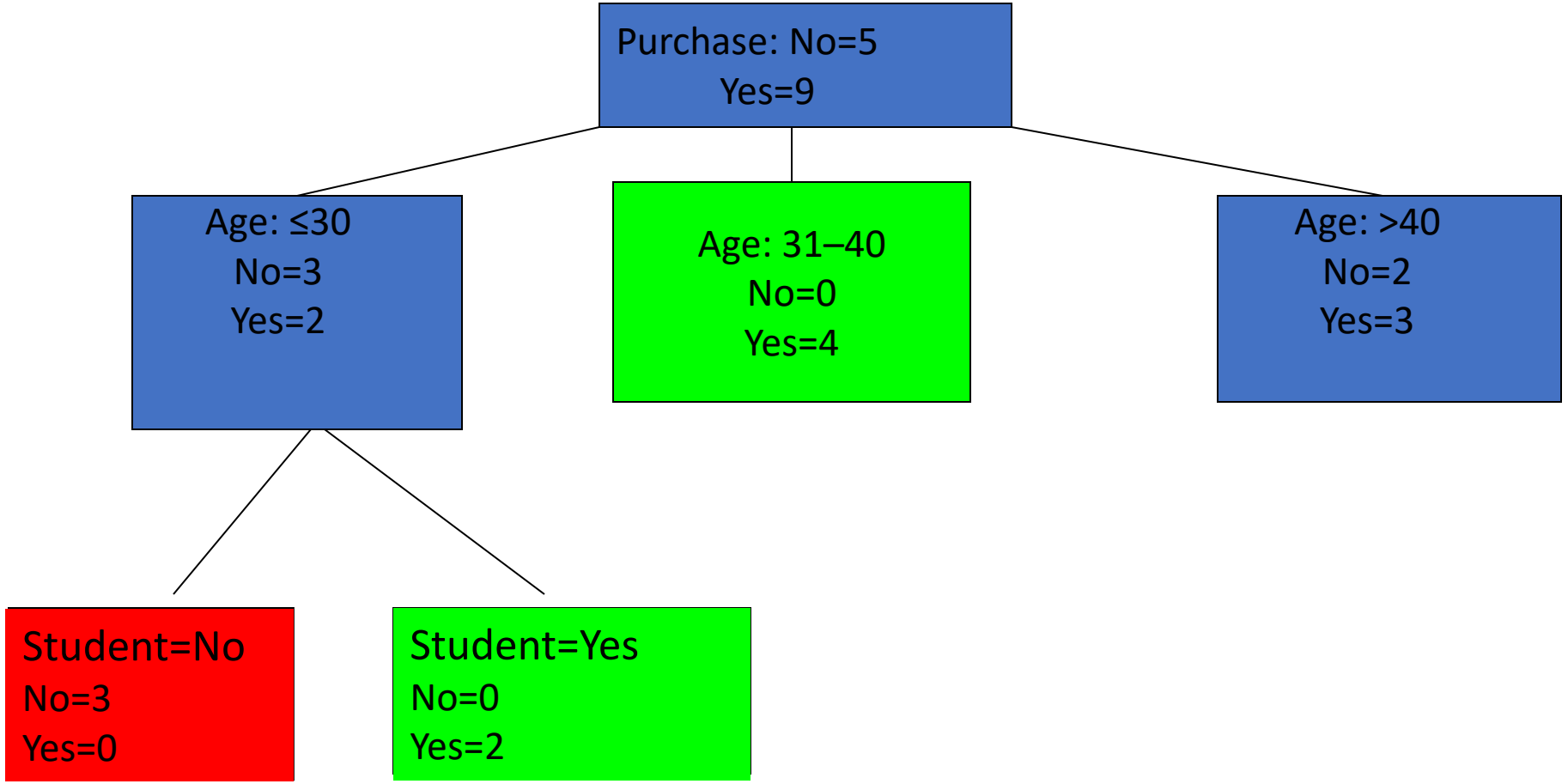
Building a Decision Tree

age	revenu	etudiant	credit	achat
<=30	eleve	non	bon	non
<=30	eleve	non	excellent	non
31-40	eleve	non	bon	oui
>40	moyen	non	bon	oui
>40	faible	oui	bon	oui
>40	faible	oui	excellent	non
31-40	faible	oui	excellent	oui
<=30	moyen	non	bon	non
<=30	faible	oui	bon	oui
>40	moyen	oui	bon	oui
<=30	moyen	oui	excellent	oui
31-40	moyen	non	excellent	oui
31-40	eleve	oui	bon	oui
>40	moyen	non	excellent	non

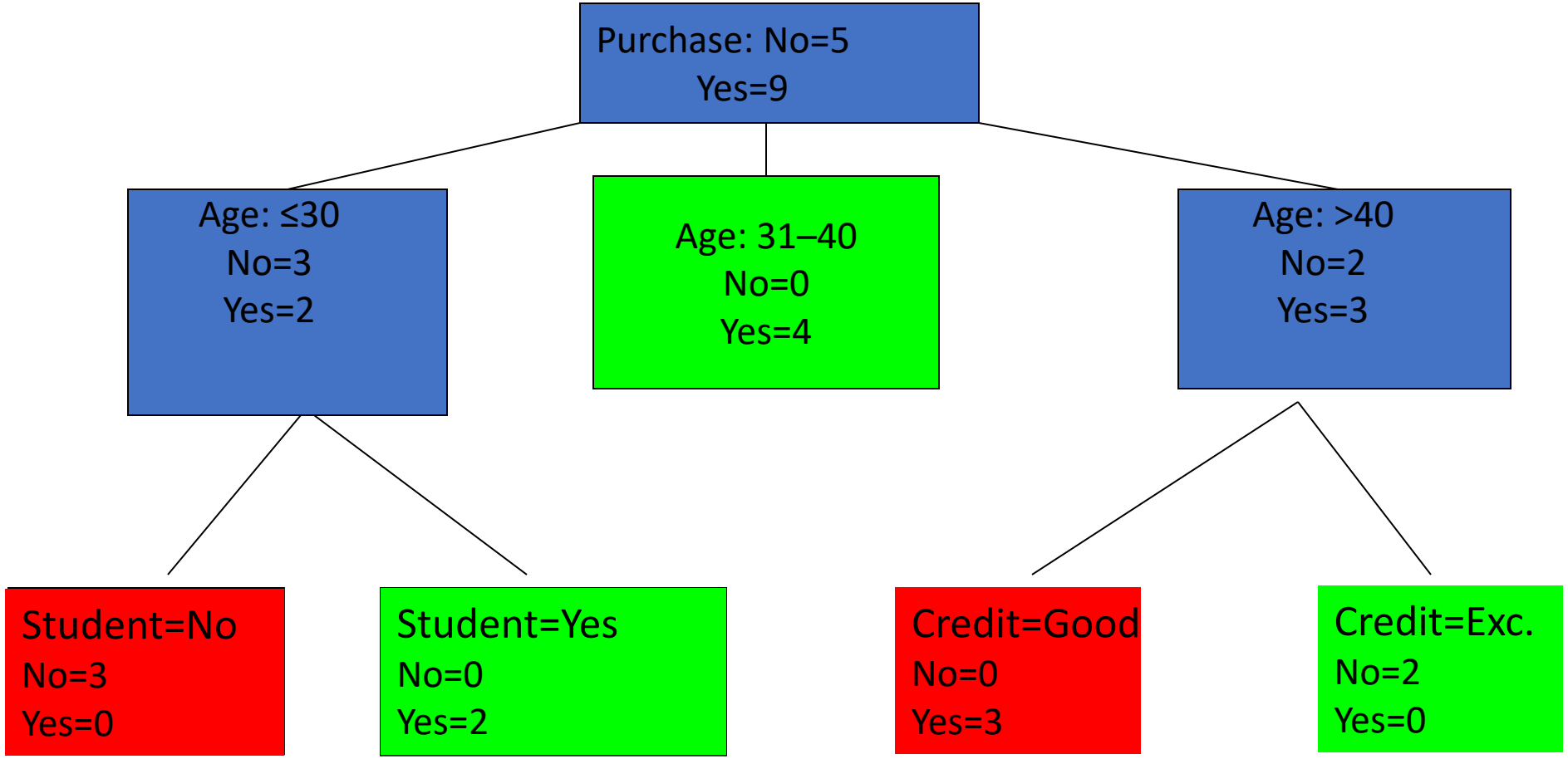
Building a Decision Tree



Building a Decision Tree



Building a Decision Tree



Building a Decision Tree

- Constructing an optimal decision tree — that is, one that minimizes the number of classification errors — is an **NP-complete problem**.
- It is therefore unrealistic always to expect a perfectly optimal tree for a given data set.
- The objective is rather to build a tree that is **correct and efficient**.
- A tree is built through a process called **Recursive Partitioning**.
  - At each step, the algorithm selects the feature and the split threshold that yield the **“purest”** (or most homogeneous) child nodes.

Building a Decision Tree

- **Recursive partitioning principle**
- The data set is recursively split into homogeneous subsets.
- Each partition aims to **minimize the impurity** of the resulting subsets.
- For a training set D containing m examples:

$$D = \{(x_i, y_i) | i = 1,2, \dots, m\}$$

- Each node splits it D into two subsets:

$$D_{left} = \{(x_i, y_i) \mid x_j < t\}, D_{right} = \{(x_i, y_i) \mid x_j \geq t\}$$

  - Where x_j is the selected variable and t is the threshold.
- The algorithm selects (x_j, t) the split that minimizes the weighted sum of **impurities** across the resulting subsets.

Building a Decision Tree

- Several automatic algorithms exist for building decision trees:
  - **ID3** (Iterative Dichotomiser 3): developed in 1986 by Ross Quinlan. It can be applied only to nominal features. It is used for classification.
  - **C4.5**: an extension of ID3 by Ross Quinlan. It can be applied to all types of features. It is used for classification.
  - **C5.0**: a commercial extension of C4.5, also by Ross Quinlan.
  - **CART** (Classification and Regression Trees): similar to C4.5 but based on other metrics. The algorithm also supports regression.

Building a Decision Tree

Algorithm	Main Splitting Criterion	Data Support and Examples	Tree Structure	Usage
ID3 (Iterative Dichotomiser 3)	**Information Gain** (based on Entropy)	**Categorical features only. Example:** Colour∈{Red, Green, Blue}; Gender∈{Male, Female}. If the data are continuous, they must be binarized manually.	**Non-binary** tree (one branch per attribute value).	Classification.
C4.5 (successor of ID3)	**Gain Ratio** (normalized entropy)	**Categorical** and **continuous/numeric**. **Example** for age (continuous), it creates a binary test (e.g., age ≤ 30 years).	**Non-binary** tree (but handles continuous variables through binary thresholds).	Classification (with pruning support).
CART (Classification and Regression Trees)	**Gini Impurity** (Classification) or **MSE** (Regression)	**Categorical** and **continuous**. **Example:** Data handling is optimized to create **simple binary tests** on any type (e.g., Income > 50 YES/NO).	Strictly **binary tree**	**Classification** and **Regression** (the modern reference).
C5.0 (Commercial Version)	**Gain Ratio** (similar to C4.5)	**All types.**	Very efficient in memory and speed. Supports **post-pruning**.	Classification (better performance than C4.5).

Building a Decision Tree

- **Impurity measures**
- The **impurity** measures the heterogeneity of a node.
- The more the samples in a node belong to the same class, the more "**pure**" the node is.
- **For Classification (Classification Trees)**
- We measure how mixed the classes are within a node.
  - Gini Impurity
  - Information Gain based on Entropy
- **For Regression (Regression Trees)**
- For a regression problem, we seek to minimize the **variance** of the target variable Y within each leaf.
  - Mean Squared Error (MSE) / Variance

Building a Decision Tree

- **Impurity measures**
- **(a) Gini Index:** Used by default for classification (CART):

$$G(t) = \sum_{i=1}^{K} p_i\,(1 - p_i) = 1 - \sum_{i=1}^{K} p_i^2$$

  - where p_i is the proportion of samples of class k in the node.
- $G(t) = 0$ → the node is **pure**.
- $G(t)$ maximal when the classes are balanced.

Building a Decision Tree

- **Impurity measures**
- **(b) Entropy (Information Gain):** A measure drawn from information theory: it quantifies uncertainty or disorder.

$$Entropy(t) = -\sum_{i=1}^{K} p_i \, log_2(p_i)$$

  - With: K = number of classes / p_i = proportion of class i
- The **information gain** is:

$$\text{Gain}(t, X_j) = Entropy(t) - \sum_{b \in \{g,d\}} \frac{|\, D_b \,|}{|\, D \,|} Entropy(D_b)$$

- The algorithm always selects the split that **maximizes** the Information Gain.

Building a Decision Tree

- **Impurity measures**
- **(c) For regression: variance**
- Impurity becomes the **variance** of the target:

$$I(t) = \frac{1}{|D_t|} \sum_{(x_i, y_i) \in D_t} (y_i - \bar{y}_t)^2$$

- The tree looks for the cut that minimizes the total variance after the split.

Building the Tree (CART algorithm)

1. **Selecting the best split**
   - For every variable and every candidate threshold, compute the impurity gain.
   - Select the split that maximizes the gain.
2. **Splitting the node** into two subsets.
3. **Recursion** on the subsets until a stopping criterion is met:
   - Maximum depth reached (max_depth)
   - Minimum number of samples in a leaf (min_samples_leaf)
   - Impurity gain < threshold
4. **Assigning a prediction** to each leaf:
   - Classification: majority class.
   - Regression: mean of the target.

Building the Tree (CART algorithm)

- **Classification example**
- Data:

Student	Study hours (X_1)	Absences (X_2)	Pass (Y)
A	2	30	No
B	3	25	No
C	4	15	No
D	5	10	Yes
E	6	5	Yes
F	8	0	Yes

- The decision tree must find **the threshold t** on the variable X_1 that separates **the classes best**.
- The CART algorithm (Classification And Regression Trees) **does not choose a threshold at random**.
- It tests **every candidate threshold located between two distinct consecutive values of X_1**, sorted in increasing order.

Building the Tree (CART algorithm)

- **Classification example**

Student	Study hours (X_1)	Absences (X_2)	Pass (Y)
A	2	30	No
B	3	25	No
C	4	15	No
D	5	10	Yes
E	6	5	Yes
F	8	0	Yes

$$G(t) = \sum_{i=1}^{K} p_i\,(1 - p_i) = 1 - \sum_{i=1}^{K} p_i^2$$

where p_i is the proportion of samples of class k in the node.

- **Step 1 — Computing the initial Gini index**
- Before any split, we have:
  - 3 students “No”
  - 3 students “Yes”

$$Gini_{initial} = 1 - (0.5^2 + 0.5^2) = 0.5$$

- This is our **starting point**: maximal impurity for two balanced classes.

Building the Tree (CART algorithm)

- **Classification example**

Student	Study hours (X_1)	Absences (X_2)	Pass (Y)
A	2	30	No
B	3	25	No
C	4	15	No
D	5	10	Yes
E	6	5	Yes
F	8	0	Yes

- **Step 2 — Testing every candidate threshold**
- The values of X_1 (study hours) are: 2, 3, 4, 5, 6, 8
- The **candidate thresholds** are placed **between two consecutive values**:

$$t \in \{2.5, 3.5, 4.5, 5.5, 7\}$$

- For each t, we compute $Gini_{split}(t)$.

Building the Tree (CART algorithm)

- **Classification example**

Student	Study hours (X_1)	Absences (X_2)	Pass (Y)
A	2	30	No
B	3	25	No
C	4	15	No
D	5	10	Yes
E	6	5	Yes
F	8	0	Yes

- **Step 2 — Testing every candidate threshold**
- **Test 1:** $t = 2.5$
- Left ($X_1 < 2.5$) → {A} → Y = No → $Gini = 0$
- Right (the rest) → {B, C, D, E, F} → 2 No, 3 Yes

$$Gini = 1 - \left(2/5\right)^2 - \left(3/5\right)^2 = 0.48$$

$$Gini_{split}(2.5) = \frac{1}{6} \times 0 + \frac{5}{6} \times 0.48 = \mathbf{0.40}$$

Building the Tree (CART algorithm)

- **Classification example**

Student	Study hours (X_1)	Absences (X_2)	Pass (Y)
A	2	30	No
B	3	25	No
C	4	15	No
D	5	10	Yes
E	6	5	Yes
F	8	0	Yes

- **Step 2 — Testing every candidate threshold**
- **Test 2:** $t = 3.5$
- Left → {A, B} → 2 No → $Gini = 0$
- Right → {C, D, E, F} → 1 No, 3 Yes

$$Gini = 1 - \left(1/4\right)^2 - \left(3/4\right)^2 = 0.375$$

$$Gini_{split}(3.5) = \frac{2}{6} \times 0 + \frac{4}{6} \times 0.375 = \mathbf{0.25}$$

t	Gini	Gain
2.5	0.40	0.10
3.5	0.25	0.25
4.5	**0**	**0.5**
5.5	0.25	0.25
7	0.40	0.1

Building the Tree (CART algorithm)

- **Classification example**

Student	Study hours (X_1)	Absences (X_2)	Pass (Y)
A	2	30	No
B	3	25	No
C	4	15	No
D	5	10	Yes
E	6	5	Yes
F	8	0	Yes

- The algorithm selects $t^* = 4.5$
  - because it **minimizes the total impurity to 0**,
- which means that **all the examples on each side of the threshold belong to the same class**.
- If study hours < 4.5 ⇒ No (likely to fail)
- Otherwise ⇒ Yes (likely to pass)

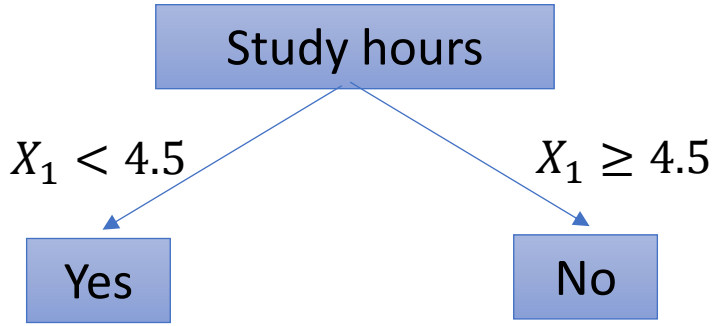
t	Gini	Gain
2.5	0.40	0.10
3.5	0.25	0.25
4.5	**0**	**0.5**
5.5	0.25	0.25
7	0.40	0.1

Building the Tree (CART algorithm)

- **Classification example**

Student	Study hours (X_1)	Absences (X_2)	Pass (Y)
A	2	30	No
B	3	25	No
C	4	15	No
D	5	10	Yes
E	6	5	Yes
F	8	0	Yes

- If study hours < 4.5 ⇒ No (likely to fail)
- Otherwise ⇒ Yes (likely to pass)



t	Gini	Gain
2.5	0.40	0.10
3.5	0.25	0.25
4.5	**0**	**0.5**
5.5	0.25	0.25
7	0.40	0.1

Building the Tree (CART algorithm)

- **Regression example**
- Data:

House	Living area (m^2)	Bedrooms	Price (€1,000s)
A	50	2	150
B	60	3	170
C	80	3	200
D	100	4	250
E	120	5	270
F	150	6	300

- A regression tree learns to **partition the data** so as to **minimize the within-node variance** of the target values.
- We want to estimate the **price of a house** (Y, in thousands of euros) from:
  - X_1: **living area (m^2)**
  - X_2: **number of bedrooms**

Building the Tree (CART algorithm)

- **Regression example**
- Data:

House	Living area (m^2)	Bedrooms	Price (€1,000s)
A	50	2	150
B	60	3	170
C	80	3	200
D	100	4	250
E	120	5	270
F	150	6	300

- **Step 1 — Building the root**
- The tree searches for the **optimal threshold t** on one variable (e.g. living area) that **minimizes the total variance** after the split.
- **Objective function**

$$\text{MSE}_{split}(t) = \frac{|\ G_{left}\ |}{|\ D\ |} Var\left(G_{left}\right) + \frac{|\ G_{right}\ |}{|\ D\ |} Var\left(G_{right}\right)$$

- The best threshold t^* is the one that **minimizes** this weighted average error.

Building the Tree (CART algorithm)

- **Regression example**
- Data:

House	Living area (m^2)	Bedrooms	Price (€1,000s)
A	50	2	150
B	60	3	170
C	80	3	200
D	100	4	250
E	120	5	270
F	150	6	300

- **Step 1 — Building the root**
- **initial variance (before any split)**
- Mean prices:

$$\bar{y} = \frac{150 + 170 + 200 + 250 + 270 + 300}{6} = 223.3$$

- Variance:

$$Var_{initial} = \frac{1}{6}\sum(y_i - \bar{y})^2 = \frac{1}{6}(5378 + 2839 + 548 + 713 + 2178 + 5867) = 2787.5$$

Building the Tree (CART algorithm)

- **Regression example**
- Data:

House	Living area (m²)	Bedrooms	Price (€1,000s)
A	50	2	150
B	60	3	170
C	80	3	200
D	100	4	250
E	120	5	270
F	150	6	300

- **Step 2 — testing the thresholds on the Living-area va**
- The sorted values of $area$ are: 50, 60, 80, 100, 120, 150
- The **candidate thresholds** are placed **between two consecutive values**:

$$t \in \{55, 70, 90, 110, 135\}$$

- For each t, we compute $MSE_{split}(t)$.

Building the Tree (CART algorithm)

- **Regression example**
- Data:

House	Living area (m²)	Bedrooms	Price (€1,000s)
A	50	2	150
B	60	3	170
C	80	3	200
D	100	4	250
E	120	5	270
F	150	6	300

- **Step 2 — testing the thresholds on the Living-area variabl**
- **Test 1:** $t = 55$
- Left: {A} → Price = 150 → Variance = 0
- Right: {B, C, D, E, F}
  - Mean = (170 + 200 + 250 + 270 + 300)/5 = 238
  - Variance ≈ 2216
- Total variance:

$$MSE_{split} = \frac{1}{6}(1 \times 0 + 5 \times 2216) \approx 1846{,}67$$

Building the Tree (CART algorithm)

- **Regression example**
- Data:

House	Living area (m^2)	Bedrooms	Price (€1,000s)
A	50	2	150
B	60	3	170
C	80	3	200
D	100	4	250
E	120	5	270
F	150	6	300

- **Step 2 — testing the thresholds on the Living-area variable**
- **Test 2:** $t = 70$
- Left: {A, B} →
  - Mean = (150+170)/2 = 160
  - Variance = 100
- Right: {C, D, E, F}
  - Mean = (200 + 250 + 270 + 300)/4 = 255
  - Variance ≈ 1325
- Total variance:

$$MSE_{split} = \frac{1}{6}(2 \times 100 + 4 \times 1325) \approx 916{,}67$$

Building the Tree (CART algorithm)

- **R**
- D
- **S**

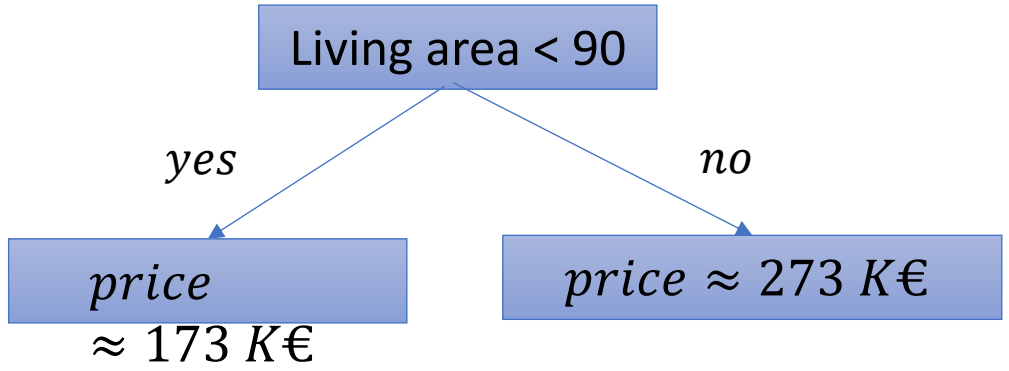
Threshold (t)	Left group (Living area ≤ t)	Left mean	Left variance	Right group	Right mean	Right variance	Weighted variance
55	{150}	150					
70	{150, 170}	160	10	300}			7
90	{150, 170, 200}	173.33	422.22	{250, 270, 300}	273.33	422.22	**422.22 (minimum)**
110	{150, 170, 200, 250}	192.5	1418.75	{270, 300}	285	225	**1020.83**
135	{150, 170, 200, 250, 270}	208	1696	{300}	300	0	**1413.33**

- **The best threshold** is the one that **minimizes the MSE**, (in this example $t^* = 90\,m^2$.
- This means that the cut at $90\,m^2$ creates two subsets that are **homogeneous** in terms of price.

Building the Tree (CART algorithm)

- **Regression example**
  - If **Living area < 90 m²**, the mean price is ≈ **€173,000**
  - If **Living area ≥ 90 m²**, the mean price is ≈ **€273,000**
- Each subgroup can in turn be split further if this **further reduces the variance**:
  - In the "Living area < 90" branch, we can test the Number of bedrooms
  - In the "≥ 90" branch, we can test Living area or Bedrooms again
- Thus, the model learns a **hierarchy of regression rules.**

House	Living area (m²)	Bedrooms	Price (€1,000s)
A	50	2	150
B	60	3	170
C	80	3	200
D	100	4	250
E	120	5	270
F	150	6	300



The Overfitting Problem

- A Decision Tree is highly flexible and can, in theory, keep splitting until every leaf is pure (i.e., contains a single sample).
- The tree then learns the training data set perfectly (including the noise), which leads to catastrophic performance on test data (high variance).

The Overfitting Problem

- To control overfitting, we rely on
  - **Pre-pruning**: limiting growth.
    - limiting the depth of the tree (max_depth),
    - Minimum number of samples:
      - min_samples_split: setting the minimum number of samples required for a node to be split.
      - min_samples_leaf: setting the minimum number of samples required for a leaf to be formed.
  - **Post-pruning**: build the full tree, then remove the least useful branches.

Advantages and drawbacks of decision trees

Aspect	Advantages	Drawbacks
Interpretability	Very **easy to understand and visualize**: the decision logic is explicit (if… then…).	Deep trees quickly become **unreadable** (loss of interpretability).
Data preparation	Little preprocessing required: no need to normalize or standardize the data.	Sensitive to **missing values** and to **noise** in the data.
Data types	Handle **numeric and categorical variables** without transformation.	Split **thresholds** can be unstable in the presence of correlated data.
Modelling	Capture **non-linear relationships** and **complex interactions** between variables.	High risk of **overfitting (overfitting)** if the tree is not properly pruned (pruning).
Speed and efficiency	Fast training and prediction on medium-sized data sets.	Perform poorly on very large or very noisy data sets without ensemble techniques.

Advantages and drawbacks of decision trees

- **When Should You Use a Decision Tree?**
- **For explainability**: when it is crucial to understand *why* a prediction was made (e.g., medicine, finance, marketing).
- **For a quick first exploration**: obtaining a baseline and insight into which variables matter.
- **On small to medium-sized data sets**.
- **When the relationships in the data are complex and non-linear**.

Advantages and drawbacks of decision trees

- Decision trees are often the **first non-linear model** taught, because they are **intuitive and interpretable**.
- But this **simplicity** comes at a price: their **instability** and their tendency **to overfit** the training data.
- This is precisely what led to the creation of **ensemble methods** (Bagging, Random Forest, Boosting), which seek to correct these weaknesses while retaining their strengths.

2.2. Bagging: Random Forests Introduction

- The decision tree is a **low-bias** model
  - it can model complex relationships.
- but with **high variance**
  - it changes radically with small variations in the training data.
- To keep the predictive power of trees while gaining stability, the idea is simple:

Why settle for a single expert (one tree) when we can have a whole **forest**?

This is the principle of **Ensemble Methods**

2.2. Bagging: Random Forests Introduction

- Ensemble methods aggregate (bring together) collections of classifiers;
- These classifiers differ from one another:
  - either by varying the data,
  - or by varying the type of classifiers;
- The final prediction is obtained by
  - majority vote for classification
  - averaging for regression.

2.2. Bagging: Random Forests — Introduction

- **Random Forest** (random forest) is an ensemble classifier that combines multiple CART-type decision trees.
- The term "**random decision forests** " was initially proposed *by Tin Kam Ho* from *Bell Labs* in *1995*.
- In 2001, a mature version of "**Random Forest"** was proposed, built on two ideas:
  - bagging
  - Random attribute selection (random space selection)

2.2. Bagging: Random Forests

A. The Bagging Concept: Bootstrap Aggregation

- The **bagging**, introduced by Leo Breiman in 1996, brings together a collection of methods.
- The term bagging is a contraction of **B**ootstrap **Agg**regat**ing**.
- It is a generic ensemble technique aimed at **reducing the variance** of unstable models (such as decision trees).

2.2. Bagging: Random Forests

A. The Bagging Concept: Bootstrap Aggregation

- The **Bagging** builds several independent decision trees using **random subsamples** of the data set.
- The key idea of the **Random Forest** is to go one step further:
  - ➢ introducing **additional randomness into the choice of variables**, in order to **decorrelate the trees**.

2.2. Bagging: Random Forests

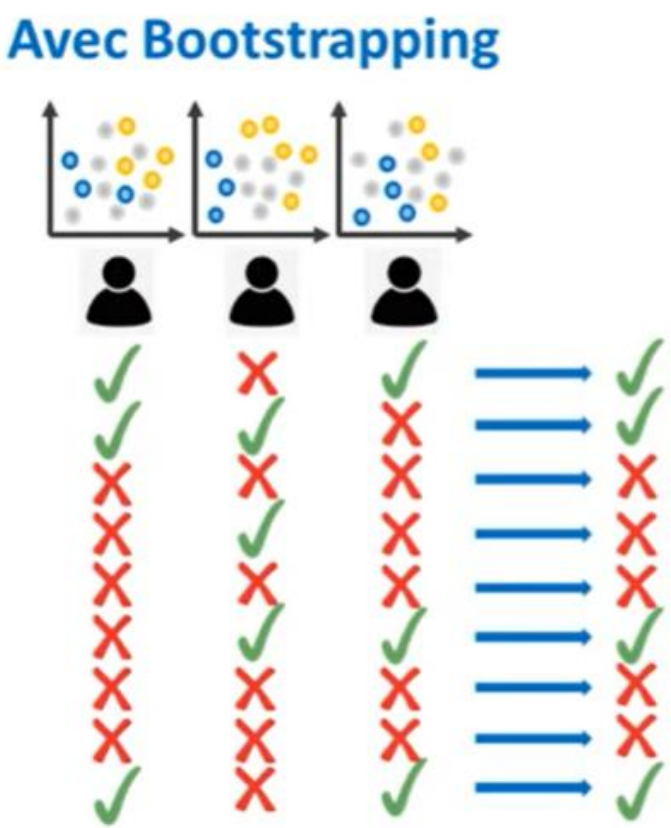
A. The Bagging Concept: Bootstrap Aggregation

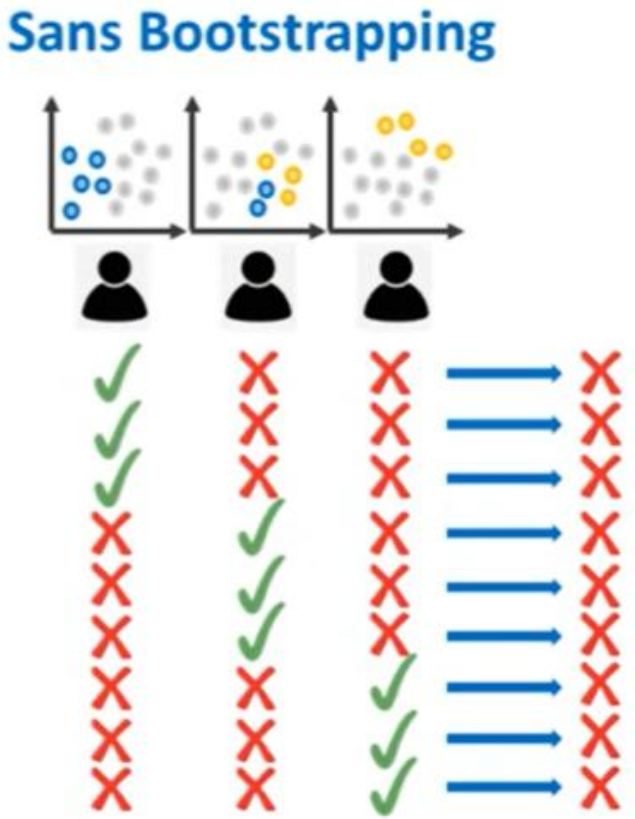
- **The Bootstrap (Resampling / sampling with replacement)**
- The main idea is to create multiple versions of the original training data set (D) using the **Bootstrap** method:
  - draw B samples $(D_1, D_2, \dots, D_B)$ from the original data set, **with replacement**.
  - Each bootstrap sample D_b has the same size m as D, but contains duplicates of some observations from the original set. It is this "with replacement" nature that guarantees diversity
- *Remark*
  - according to studies, **37%** of the original observations are never selected; these points are called **Out-Of-Bag** samples.

2.2. Bagging: Random Forests

A. The Bagging Concept: Bootstrap Aggregation

- **The Bootstrap (Resampling)**





2.2. Bagging: Random Forests

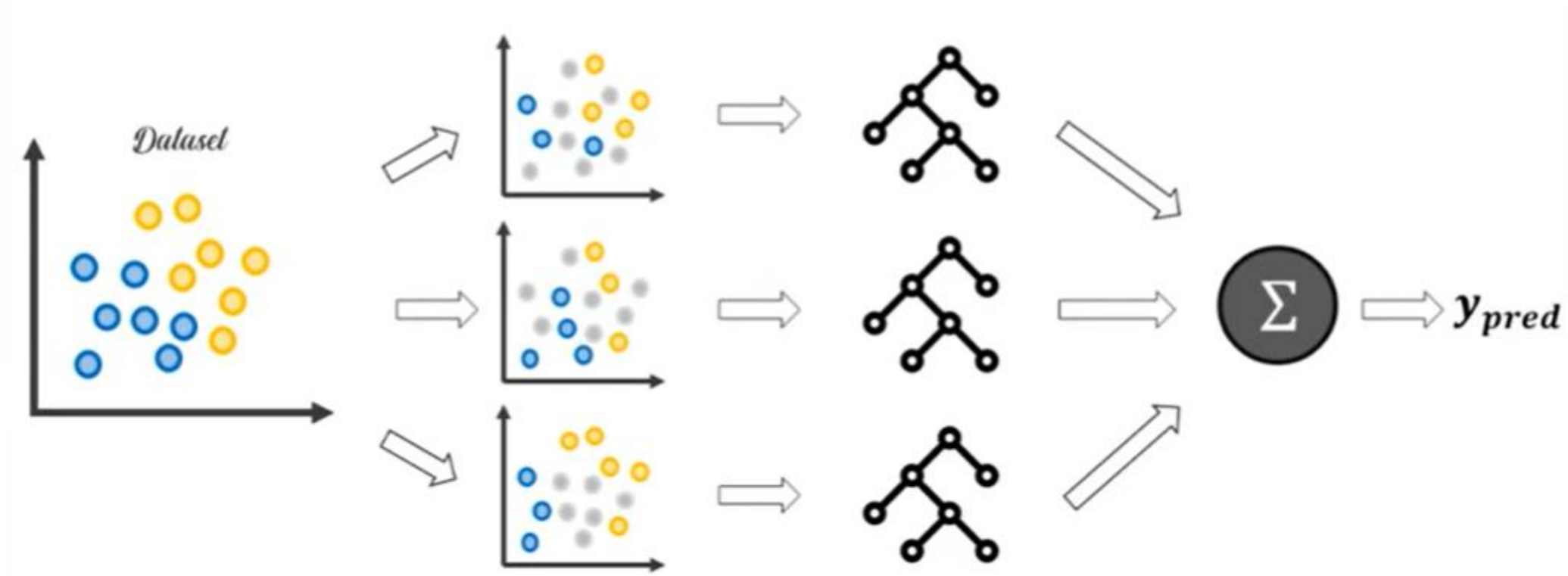
A. The Bagging Concept: Bootstrap Aggregation

- **The Aggregation (Aggregating)**
- Once the B data sets have been created, the base model (a *base learner*) is trained on each of them, producing a collection of models $f_1, f_2, \ldots, f_B$.
- The aggregated prediction $\hat{y}$ is obtained through:
  - **Classification: Majority Voting** (aggregation by *mode*).
  - **Regression: Averaging** of the predictions.

2.2. Bagging: Random Forests

A. The Bagging Concept: Bootstrap Aggregation

- **The Aggregation (Aggregating)**



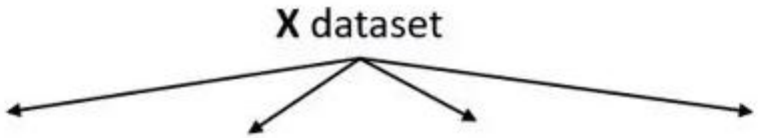
2.2. Bagging: Random Forests

B. Overall Operation of a Random Forest

- **Bootstrapping the data:**
  - Create M subsamples (with replacement) from the training set.
  - Train a decision tree on each sample.

2.2. Bagging: Random Forests

B. Overall Operation of a Random Forest



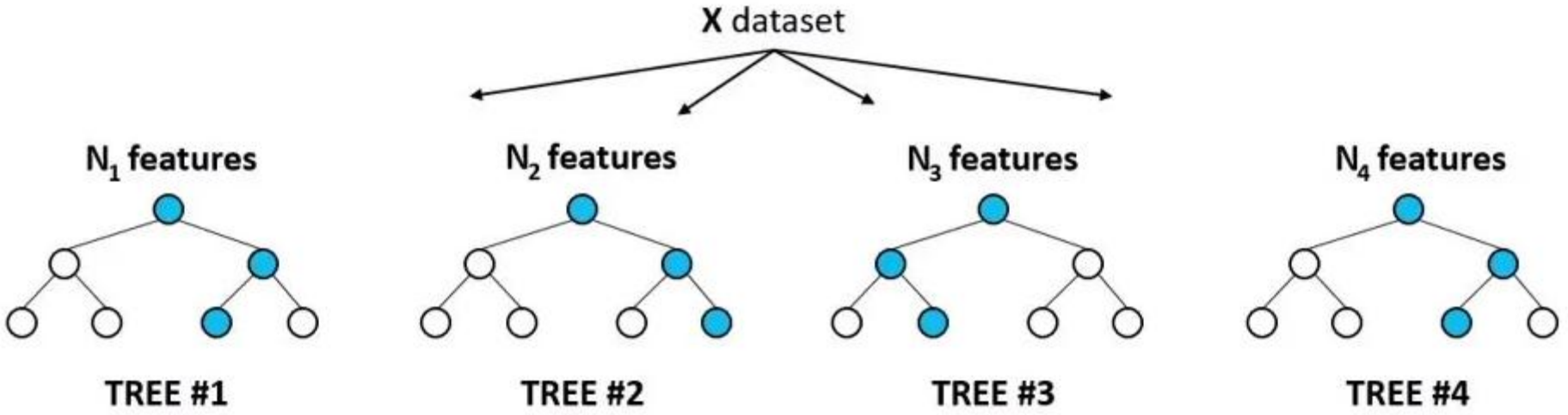
2.2. Bagging: Random Forests

B. Overall Operation of a Random Forest

- **Bootstrapping the data:**
  - Create M subsamples (with replacement) from the training set.
  - Train a decision tree on each sample.
- **Random selection of variables:**
  - At **each node** of each tree,
    we select **a random subset of features** (e.g., $\sqrt{p}$ among p features).
  - We select the **best split** among these variables.

2.2. Bagging: Random Forests

B. The Bagging Concept: Bootstrap Aggregation



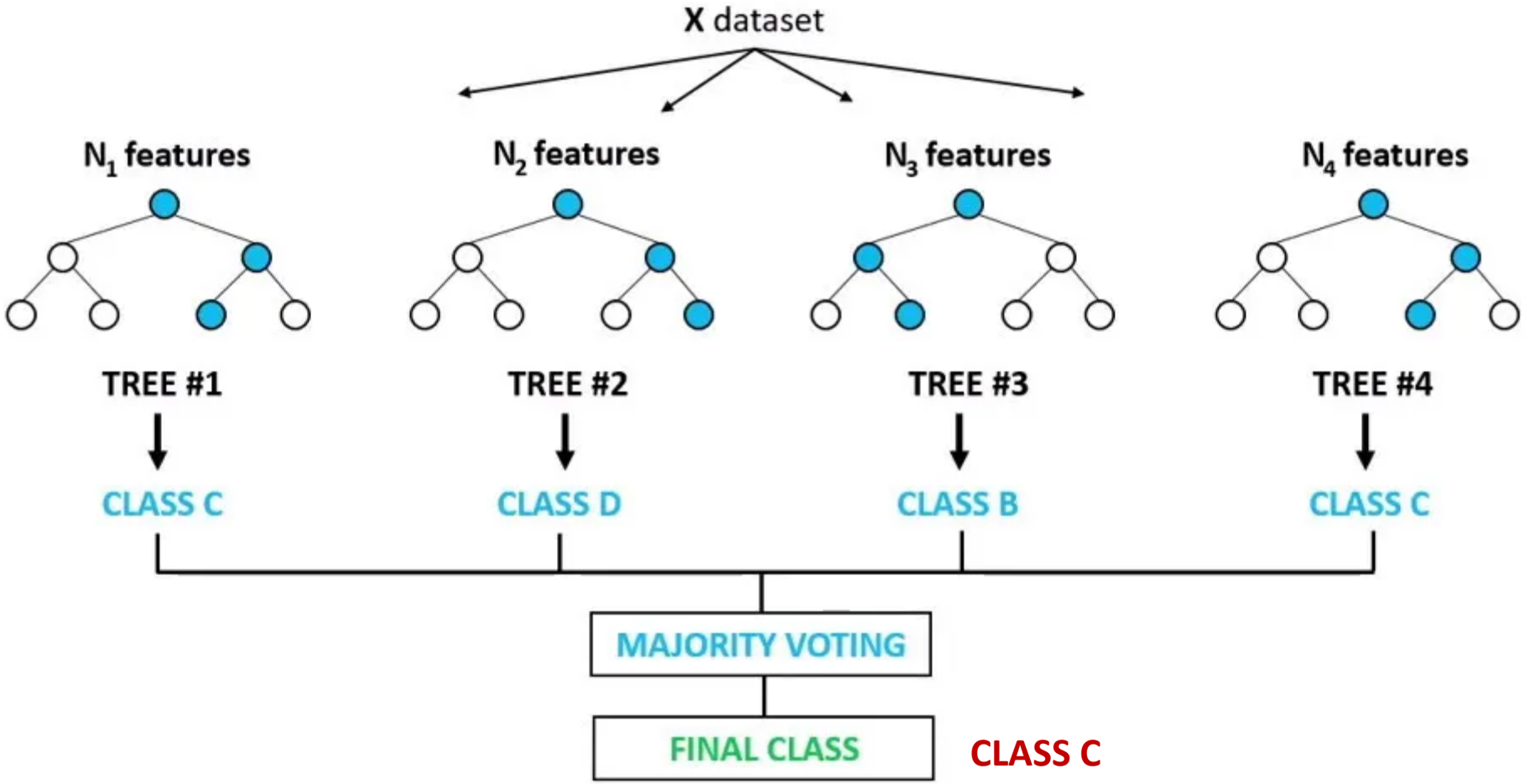
2.2. Bagging: Random Forests

B. Overall Operation of a Random Forest

- **Bootstrapping the data:**
  - Create M subsamples (with replacement) from the training set.
  - Train a decision tree on each sample.
- **Random selection of variables:**
  - At **each node** of each tree,
    we select **a random subset of features** (e.g., $\sqrt{p}$ among p features).
  - We select the **best split** among these variables.
- **Aggregating the results:**
  - For **regression**: the predictions are averaged.
  - For **classification**: majority vote.

2.2. Bagging: Random Forests

B. Overall Operation of a Random Forest



2.2. Bagging: Random Forests

B. Overall Operation of a Random Forest

- The Random Forest introduces a twofold randomization to guarantee that the trees are decorrelated:

1. **Randomization of the Sample (Bagging)**
   - Each tree is trained on a bootstrap subsample of the data (variance reduction).
2. **Randomization of Features (Feature Randomness)**
   - At each node of each tree, the algorithm considers only a random subset of m features to determine the best split.

- *Rule of thumb* :
  - For Classification: $m \approx \sqrt{p}$ (where p is the total number of features).
  - For Regression: $m \approx p/3$.

2.2. Bagging: Random Forests

C. The Advantages of Random Forests

- **1. Overfitting Reduction**
- By averaging overfitted yet decorrelated trees, the Random Forest is far more **robust** to noise and overfitting than any single tree.
- **2. Out-Of-Bag Error (OOB Error)**
- These **OOB** samples can be used as an **internal validation set:**
  - For each observation xi, use only the trees that have **never** seen it to predict its target.
  - Compute the error over this set of predictions.
- The OOB error is a **nearly unbiased** estimate of the generalization error on unseen data, without the need for a separate validation set.

2.2. Bagging: Random Forests

C. The Advantages of Random Forests

- **3. Feature Importance**
- The Random Forest provides a reliable feature-importance score by quantifying the **total mean decrease in impurity** (Gini or MSE) that a feature brings about across **all** nodes of **all** the trees in the forest.
- It is a powerful tool for *Feature Selection*, as it identifies the variables that contribute most to the accuracy of the ensemble.

2.3. Boosting

Introduction

- The **Boosting** is the second major family of ensemble methods. It was introduced by Freund et al. (1996).
- Unlike Bagging (independent, parallel training), Boosting is a **sequential** and **adaptive** process in which each new model is trained to correct the errors (residuals) made by the ensemble of previous models.
- It relies on the following idea:

Improving a model's performance by **iteratively correcting its errors**,

by combining several **weak models** (weak learners) to form a **strong model** (strong learner).

2.3. Boosting

A. The Concept of Boosting

- **What is a “weak learner”?**
- A **weak learner** (*weak learner*) is a model that performs **barely better than chance** on a given problem.
  In other words, its error rate is slightly below 50% (in binary classification).
- **What is a “strong learner”?**
- A **strong learner** (*strong learner*) is a model capable of achieving **very high accuracy** on the data.
  → Example: an ensemble of combined trees that reaches **95%** accuracy.

2.3. Boosting

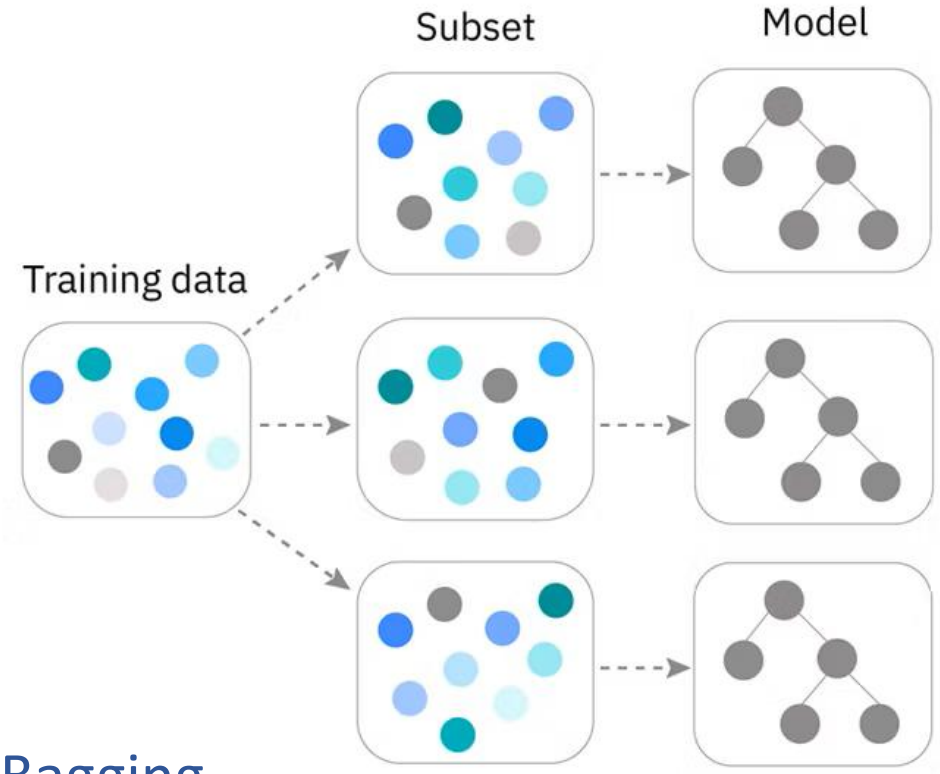
A. The Concept of Boosting

- **Conditions for success**:
  - The base classifiers must be “weak”.
  - The base classifiers (w.l) must be dependent,
    - That is, the training set X_{t+1} of the classifier h_{t+1} is built based on the performance of h_t on the set X_t
  - Weak learners must have low variance and high bias

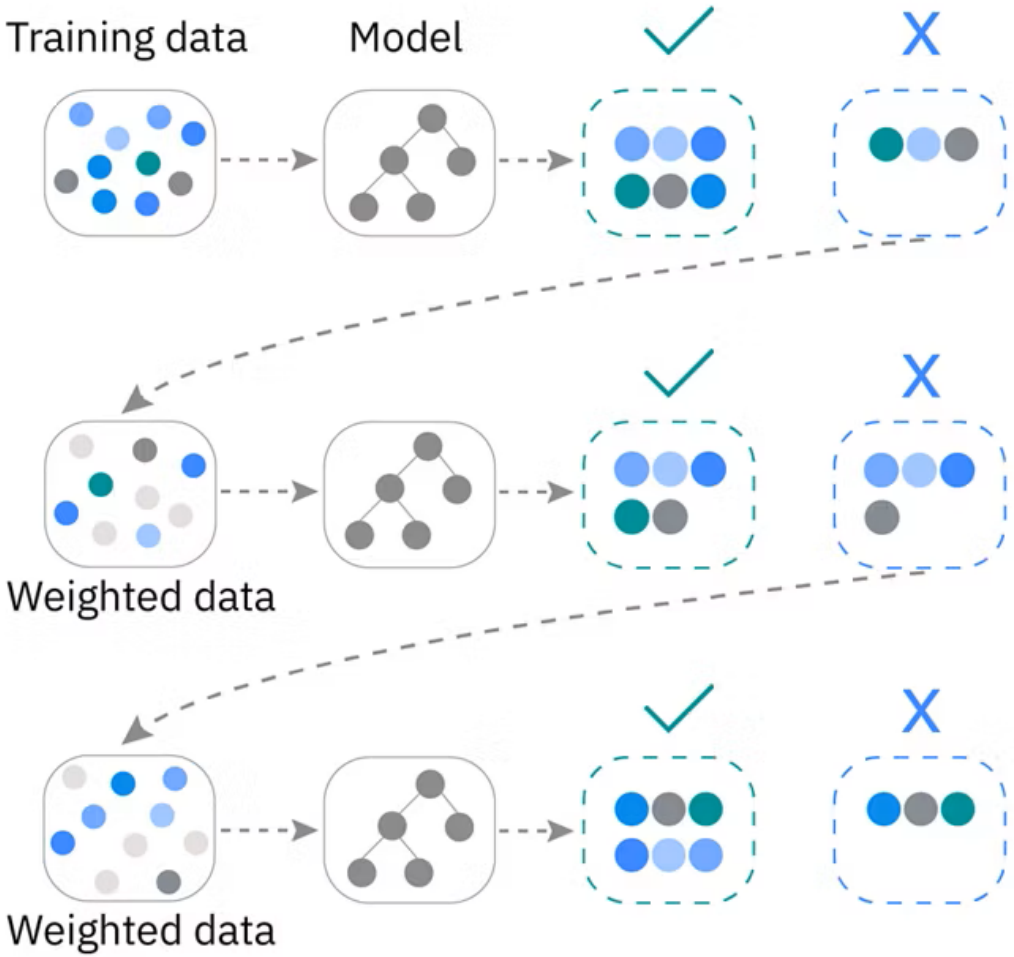
❑ The boosting algorithm can provably build a classifier with an error rate $< \varepsilon$

2.3. Boosting

A. The Concept of Boosting



Bagging

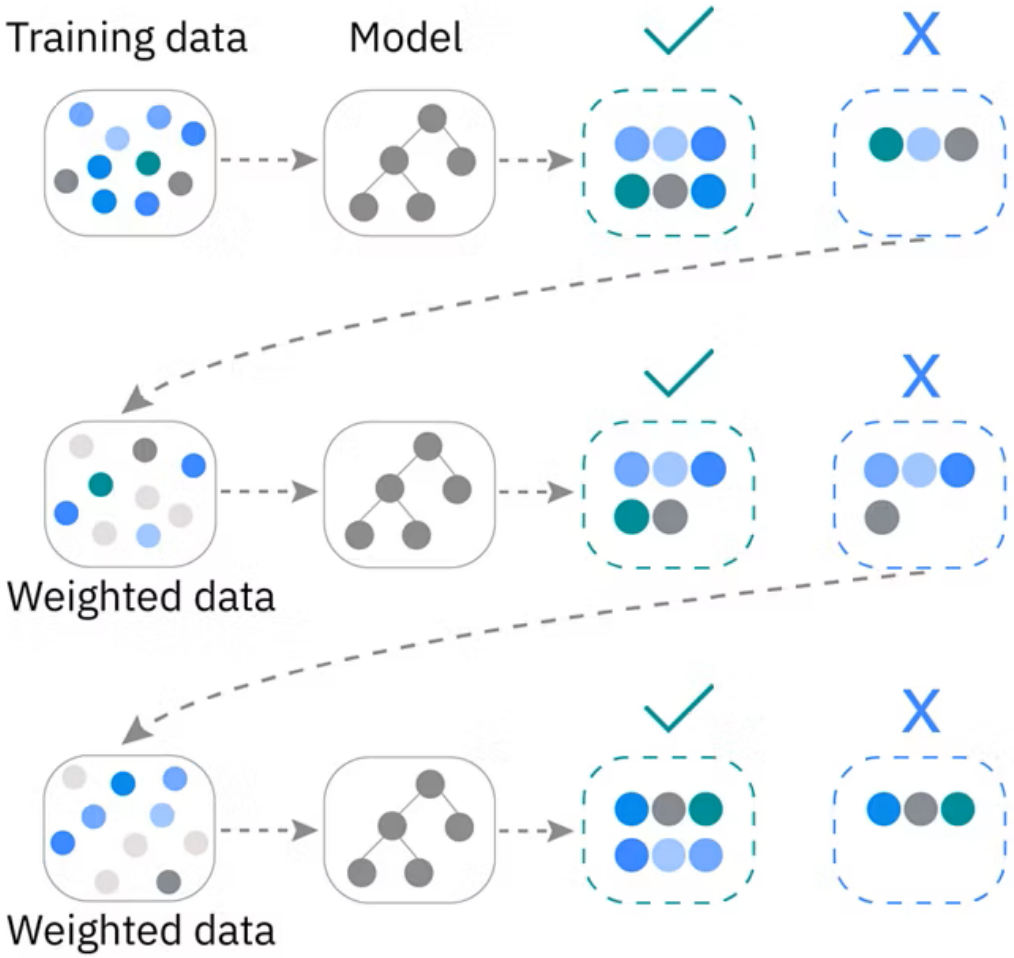


Boosting

2.3. Boosting

A. The Concept of Boosting

- General principle of Boosting
- Initialize a simple model (e.g., a small decision tree).
- Compute the prediction errors.
- Give more weight to the mispredicted examples.
- Train a new model to correct these errors.
- Repeat until convergence.



Boosting

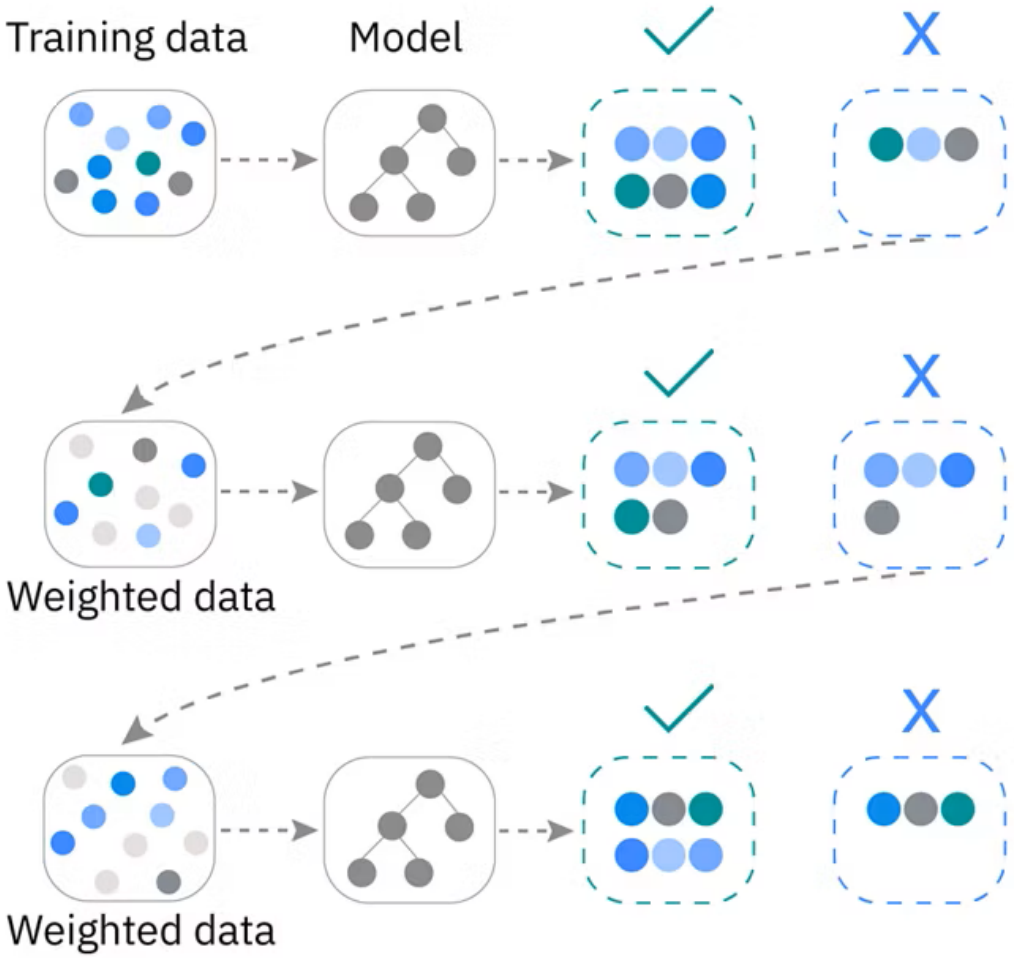
2.3. Boosting

A. The Concept of Boosting

- General principle of Boosting

$$\hat{y}(x) = \sum_{m=1}^{M} \alpha_m \, h_m(x)$$

- $h_m(x)$: prediction of weak model no. m
- α_m: weight assigned to each model
- M: number of iterations



Boosting

2.3. Boosting

B. Types of Boosting

- Boosting algorithms may differ in how they create and combine weak learners over the sequential process.
- The five most common boosting methods are the following:
  - **Adaptive Boosting, or AdaBoost**
  - **Gradient Boosting**
  - **Extreme Gradient Boosting, or XGBoost**
  - **LightGBM**
  - **CatBoost**.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
- This is the historic algorithm that formalized the concept of Boosting.
- It was introduced by Freund & Schapire (1997).
- The **“weak learners”** in AdaBoost are usually decision trees with only **2 branches** and **2 leaves** (also called stumps)
  - but other types of classifiers can be used.
- each weak learner is trained on a **weighted data set**.
- The weights are updated after each iteration:
  - the misclassified examples see their weight increase.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
- **1- Initialization**: All the examples start with the same weight:

$$w_i = \frac{1}{N}, \forall i$$

- **2- Iteration** $(for\ m\ =\ 1\ to\ M)$:
  a) Train a weak learner $h_m(x)$ on the weighted data.
  b) Compute the weighted error rate:

$$\varepsilon_m = \frac{\sum_{i=1}^{n} w_i . I[y_i \neq h_m(x_i)]}{\sum_{i=1}^{n} w_i}$$

With:

- n: total number of examples.
- w_i : the weight of example i.
- y_i : the label of example i.
- $h_m(x_i)$: the prediction of model m for example i.
- $I[y_i \neq h_m(x_i)]$: **indicator function**, equal to 1 if the prediction is wrong, and 0 otherwise.
- ε_m quantifies the quality of the **last weak learner** as a function of the **weighted difficulty** of the data.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
  - c) Compute the model weight:

$$\alpha_m = \frac{1}{2} ln\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$$

1. **Scaling:** It converts the ratio (which can range from 0 to infinity) into a more stable value suitable for a weight (α_m ranges from $-\infty$ to $+\infty$).
2. **Penalty/Reward:** It amplifies the difference around the crucial reference point ($\varepsilon_m = 0.5$).

- α_m : determines the importance (the weight) of weak model h_m in the final model.
- The ratio $\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$ measures how much better the model is than chance:
  - ε_m is the weighted error rate of h_m.
  - $(1-\varepsilon_m)$ is the weighted accuracy (or weighted success rate) of model h_m .

2.3. Boosting

B. Types of Boosting

Scenario	Weighted Error Rate (ε_m)	Value of α_m	Interpretation
High-performing Model	$\varepsilon_m \approx 0$ (very small)	$\alpha_m \to +\infty$ (very large and positive)	The tree did very well, even on the hard cases. It is extremely reliable and receives a very high weight in the final vote.
Random Model	$\varepsilon_m = 0.5$	$\alpha_m = 0$	The model is equivalent to making a random prediction. The ratio $\frac{1-0,5}{0,5} = 1$, and $\ln(1) = 0$. The model contributes no useful information and is ignored in the final vote.
Negative Model	$\varepsilon_m > 0.5$ (worse than chance)	$\alpha_m < 0$ (negative)	The model is judged worse than chance. The ratio is below 1, giving a negative logarithm. The algorithm assigns the model a negative weight, which means it will take the inverse of its prediction in the aggregation.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
  - c) Compute the model weight:

$$\alpha_m = \frac{1}{2} ln\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$$

  - d) Update the example weights:

$$w_i \leftarrow w_i . e^{-\alpha_m y_i h_m(x_i)}$$

then normalization:

$$w_i \leftarrow \frac{w_i}{\sum_j w_j}$$

- the exponential term $e^{-\alpha_m y_i h_m(x_i)}$) guarantees that the errors ($y_i \neq h_m(x_i)$) see their weight increase.
- **Correctly classified case** ($y_i = h_m(x_i)$) **:**
  - If the prediction is correct, then the product $y_i . h_m(x_i) = 1$.
  - The new weight is $w_i \leftarrow w_i . e^{-\alpha_m}$. Since α_m is positive for a good model, $. e^{-\alpha_m} < 1$, the weight w_i **decreases**.
- **Misclassified case** ($y_i \neq h_m(x_i)$) **:**
  - If the prediction is wrong, then the product $y_i . h_m(x_i) = -1$.
  - The new weight is $w_i \leftarrow w_i . e^{\alpha_m}$. Since α_m is positive, e^{α_m} >1, so the weight w_i **increases exponentially**.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
  - c) Compute the model weight:

$$\alpha_m = \frac{1}{2} ln\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$$

  - d) Update the example weights:

$$w_i \leftarrow w_i . e^{-\alpha_m y_i h_m(x_i)}$$

then normalization:

$$w_i \leftarrow \frac{w_i}{\sum_j w_j}$$

- **Final combination:**

$$H(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m \, h_m(x)\right)$$

- AdaBoost builds **a sequence of weak classifiers** $h_1(x), h_2(x), \ldots, h_M(x)$ (usually small decision trees, *stumps*).
- AdaBoost lets all the models vote, but some carry more voice than others (depending on their performance).
- Each model has an influence proportional to its reliability α_m:
  - A very accurate model (low error) → large α_m → strong vote.
  - A mediocre model → small α_m → weak vote.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**

Hyperparameter	Effect on the model	Practical advice
Number of weak classifiers (M)	The larger (M) is, the more the model can reduce the bias, but it risks overfitting if the data set is noisy.	Start with 50–200. Use cross-validation to detect the saturation point.
Learning rate	This hyperparameter does not appear in the classical AdaBoost algorithm but is crucial in modern implementations (such as Scikit-learn). It acts as a regularizer.	• Default = 1: classical AdaBoost. • Try 0.1 to 0.5 for noisy data.
Weak model used (stumps).	Affects the local learning capacity. Stumps (depth 1) provide robustness and limit overfitting.	Use shallow trees (depth ≤ 3).
Depth of the weak classifier	Greater depth → higher overfitting risk, but it captures complex interactions.	In practice: depth=1 to 3.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
- **Advantages of AdaBoost**
- **High Accuracy and Theoretical Foundation:** It is the first algorithm proved capable of **turning** weak learners into strong learners. It is often highly performant.
- **Flexibility of the Weak Learner:** Although *stumps* are usually used, AdaBoost can in principle boost any classification algorithm, not only trees.
- **Fast Implementation (with Stumps):** Training *stumps* is extremely fast, making the overall algorithm efficient (faster than Random Forests, which require deep trees).

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
- **Drawbacks and Critical Limitations**
- **Extreme Sensitivity to Noise and Outliers:** This is the major weakness of AdaBoost (the weight update is **exponential**). It is sensitive to noise and outliers, because the weights of such points grow exponentially. To mitigate this, one reduces learning_rate or uses smoother variants (GentleBoost, LogitBoost).
- **Risk of Overfitting (if uncontrolled):** Without careful tuning of the trio n_estimators, base_estimator, and learning_rate, the model will overfit.
- **Sequential Nature (Not Parallelizable):** The process is inherently sequential. Tree hm cannot be trained before hm−1 has finished. This makes it slower than Bagging (Random Forest), which is perfectly parallelizable.

2.3. Boosting

B. Types of Boosting

- **AdaBoost (Adaptive Boosting)**
- AdaBoost marks **the historical starting point** of the boosting concept.
- Its main strengths lie in its mathematical simplicity and its adaptive principle.
- However, its main drawback — **sensitivity to noise and outliers** — led to the design of new, more robust and more performant variants.

2.3. Boosting

B. Types of Boosting

- **Gradient Boosting Machine (GBM)**
- In AdaBoost, each new classifier corrects the errors of the previous ones by **reweighting the mispredicted examples**.
  This adaptive logic is powerful, but AdaBoost has two limitations:
  - It **does not explicitly optimize a cost function**.
  - It **strongly amplifies noise**, because gross errors receive an exponential weight.
- **Gradient Boosting** generalizes this idea:
  instead of reweighting the examples, it **adjusts the model directly by following the gradient of the loss function**.
- In other words: AdaBoost is a special case of Gradient Boosting using an exponential loss.

2.3. Boosting

B. Types of Boosting

- **Gradient Boosting Machine (GBM)**
- The main idea of GBM, introduced by **Jerome Friedman**, is to transform the sequential optimization problem:
  - **AdaBoost :** The new learner h_m is trained on a **reweighted** version of the target data y_i . It focuses on sample *errors*.
  - **GBM:** The new learner h_m is trained to directly predict the **uncorrected errors (residuals)** of the previous cumulative model F_{m-1}. It focuses on prediction *errors*.
- The objective at each iteration m is to find a weak learner h_m such that, when added to the current model F_{m-1} , it minimizes the loss function $L(y, F(x))$ as quickly as possible:

$$F_m(x) = F_{m-1}(x) + h_m(x) \text{ such that } L(y, Fm(x)) \; is \; minimized$$

2.3. Boosting

B. Types of Boosting

- **Gradient Boosting Machine (GBM)**
- The GBM builds the model F_m sequentially. At each step m, it seeks to add a new weak tree h_m that corrects the remaining errors F_{m-1} .
  1. **Initial Model:** $F_0(x)$ is a constant prediction (for example, the mean of the y in regression).
  2. **Computing the Pseudo-Residuals:** For each observation i, we compute the **pseudo-residual** r_{im} . It is not simply the error $(y_i - \hat{y}_i)$, but the **negative gradient** of the cost function with respect to the previous prediction:

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i)}{\partial F(x_i)}\right]_{F=F_{m-1}}$$

2.3. Boosting

B. Types of Boosting

- **Gradient Boosting Machine (GBM)**
  3. **Training the New Tree:** We train a new regression tree (a *weak learner*) h_m so that it predicts these pseudo-residuals r_{im}.
  4. **Updating the Ensemble:** We add the new tree to the ensemble, weighting it by a **learning rate v (Learning Rate)** :

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

  - The **Learning Rate (ν)** (or *shrinkage*) is the most important hyperparameter: it controls each tree's contribution in order to avoid overfitting.

2.3. Boosting

B. Types of Boosting

- **Gradient Boosting Machine (GBM)**
- Some loss functions, by task type

Task type	Loss function (L(y, \hat{y}))	Example
Regression	$\frac{1}{2}(y_i - \widehat{y_i})^2$	Price or temperature prediction
Binary classification	$\log(1 - e^{-2y_i\widehat{y_i}})$	Spam / not spam
Multi-class classification	Cross-entropy	Image recognition

2.3. Boosting

B. Types of Boosting

- **Gradient Boosting Machine (GBM)**
- Gradient Boosting is an **elegant yet costly formulation**:
  each tree is trained sequentially, and computing the gradients can be slow on large data sets.
- To overcome these limitations, several optimized libraries have been developed:
  - **XGBoost (2016)**: introduction of regularized and parallelized computation.
  - **LightGBM (2017)**: leaf-wise growth to speed up computation.
  - **CatBoost (2018)**: native handling of categorical variables and built-in regularization.

2.3. Boosting

B. Types of Boosting

- **XGBoost (Extreme Gradient Boosting)**
- **XGBoost** is an **optimized** and **regularized** implementation of the GBM. It is the algorithm that dominated competitions (such as Kaggle) for years.
- **Key Differences from the standard GBM:**

1. **Built-in Regularization:** This is its greatest strength. XGBoost adds **L1 (Lasso)** and **L2 (Ridge)** regularization terms to its cost function. The penalty applies to the complexity of the tree (number of leaves) and to the magnitude of the leaf weights, which combats **overfitting** natively.

2.3. Boosting

B. Types of Boosting

- **XGBoost (Extreme Gradient Boosting)**
- **XGBoost** is an **optimized** and **regularized** implementation of the GBM. It is the algorithm that dominated competitions (such as Kaggle) for years.
- **Key Differences from the standard GBM:**

2. **Speed Optimization (Parallelization):** Although the trees are built sequentially, XGBoost parallelizes the *construction of each tree*. It uses sorted data "blocks" to evaluate the gains of all candidate split thresholds in parallel very quickly.

2.3. Boosting

B. Types of Boosting

- **XGBoost (Extreme Gradient Boosting)**
- **XGBoost** is an **optimized** and **regularized** implementation of the GBM. It is the algorithm that dominated competitions (such as Kaggle) for years.
- **Key Differences from the standard GBM:**

3. **Handling Missing Values:** XGBoost has an internal mechanism to learn the "best path" (left or right) for missing values (NaN) during training, thereby avoiding complex preprocessing (imputation).

2.3. Boosting

B. Types of Boosting

- **LightGBM (Light Gradient Boosting Machine)**
- **LightGBM** (by Microsoft) is another GBM implementation, designed to be **extremely fast** and efficient on **very large data sets** (Big Data).
- **CatBoost (Categorical Boosting)**
- **CatBoost** (by Yandex) is the third major implementation, whose main strength is its revolutionary handling of **categorical features**.

2.4. Stacking and Blending

Introduction

- So far, we have covered:
  - **Bagging (Bootstrap Aggregating)**: combines several independent models (like the trees in a random forest).
  - **Boosting**: builds models sequentially to correct the errors of the previous ones.
  - These are **homogeneous** ensemble methods, where the strength comes from aggregating multiple versions of the same algorithm (usually trees).
- But there exists a **third family** of ensemble methods: **heterogeneous methods.**

2.4. Stacking and Blending

Introduction

- The fundamental principle is no longer merely to reduce
  - the variance (Bagging)
  - or the bias (Boosting),
- but to start from the assumption that a **combination of fundamentally different models** will outperform any of these models taken individually.
- This is **Stacking**: the art of **meta-model** learning.

2.4. Stacking and Blending

B. The Concept of Stacking

- No learning algorithm is optimal on every type of data.
- Thus:
  - a **logistic regression** may generalize better on linear data,
  - a **tree** captures non-linear interactions well,
  - …
- The idea behind *Stacking* is to **blend several models of different natures** in order to create a **meta-model** (or *meta-learner*) that learns **when and how to trust** each of the base models.

2.4. Stacking and Blending

B. The Concept of Stacking

- Stacking is a multi-level architecture. In its simplest version (2 levels):
- **Level 0 (Base Learners)** : A collection of **heterogeneous** (diverse) models.
- **Level 1 (Meta-Learner)** : A single model (the meta-model) that takes the outputs of Level 0 as **input features** (meta-features).

2.4. Stacking and Blending

B. The Concept of Stacking

- **Level 0 (Base Learners):**
- These are the first models trained on the original data.
- Examples:
  - Linear regression
  - Decision tree
  - SVM
  - k-NN
  - Simple neural network
- Each model produces a prediction:

$$\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_K$$

2.4. Stacking and Blending

B. The Concept of Stacking

- **Level 1 (Meta-Learner):**
- A single model (the *meta-model*) that takes the outputs of Level 0 as **input features** (meta-features).
- We no longer predict from the original variables X, but from the **predictions of the base models**.
- Thus, we create a new data set:

$$Z = [\hat{y}_1, \hat{y}_2, . , .. \hat{y}_K]$$

- and the **meta-model** learns to combine these predictions in order to predict the final output (target)$\hat{y}_{\text{final}}$.

$$\hat{y}_{\text{final}} = \text{f}(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{\text{K}})$$

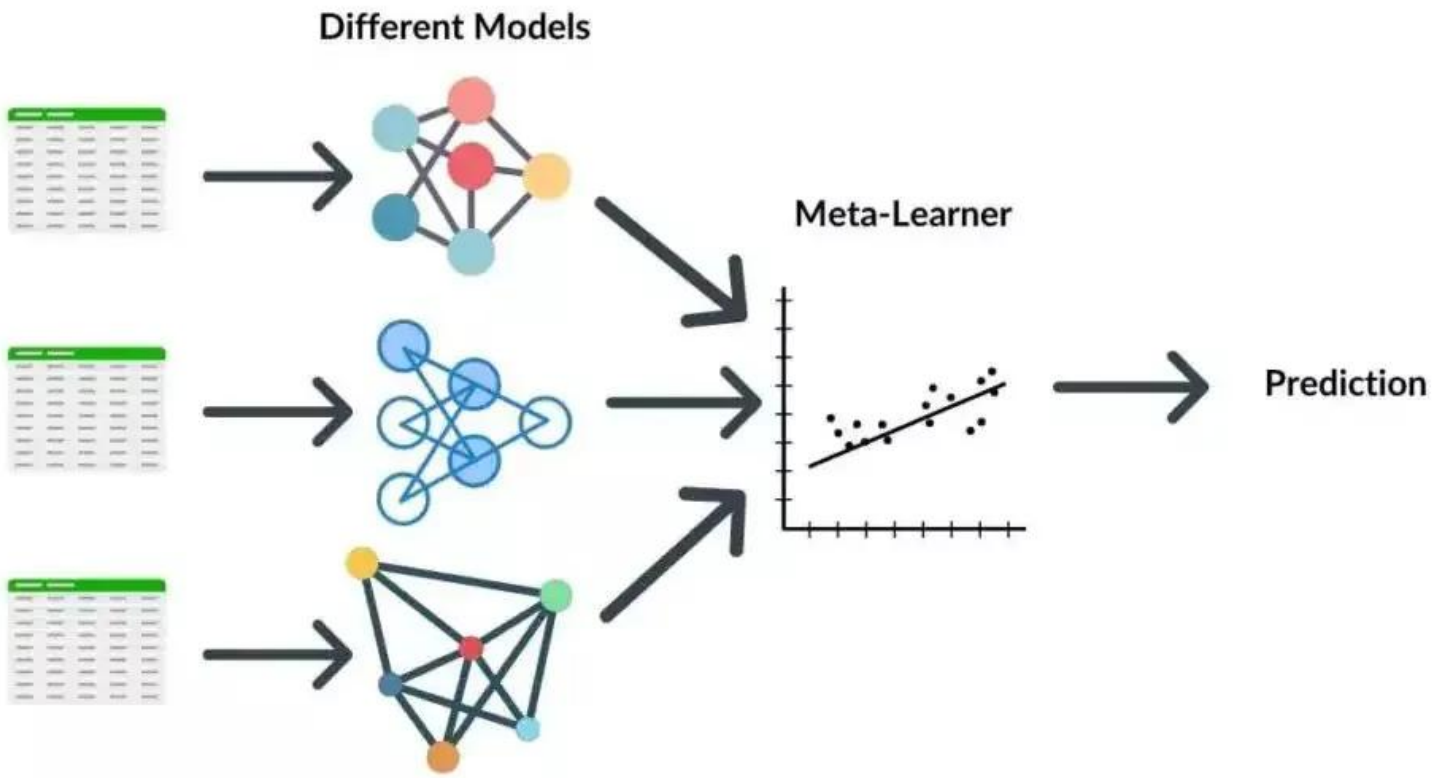
2.4. Stacking and Blending

B. The Concept of Stacking

- **Level 1 (Meta-Learner):**
- Examples of meta-models:
  - Linear regression (often used to combine regressions)
  - Logistic regression (for classification)
  - More complex models: Random Forest, Gradient Boosting, etc.
- The goal of the meta-model (Level 1) is not to average the votes, but to learn the **error patterns** of the base models.
- A **simple** meta-model is almost always preferable.
  - A complex meta-model risks **overfitting** the predictions (and the errors) of the base models

2.4. Stacking and Blending

B. The Concept of Stacking



2.4. Stacking and Blending

B. The Concept of Stacking

- **Data Leakage**
- **The Trap:** Training the Meta-Model (Level 1) using predictions (meta-features) generated by the Base Models (Level 0) **on the same data set** Dtrain as the one used to train those base models.
- **The Failure Mechanism:** The Level 0 predictions are **compromised by overfitting**. They are “too perfect” because they have memorized the noise and the specificities of Dtrain.
- **The Consequence:** The Meta-Model learns to exploit this **noise** (the overfitting of N0) instead of its **capacity to generalize**.
- **The Result: Massive overfitting** of the Meta-Model, making it unable to perform on unseen data.

2.4. Stacking and Blending

B. The Concept of Stacking

- **The Strategy: K-Fold Stacking (Out-of-Fold generation – OOF)**
- **The Principle:** The training set Xmeta (for Level 1) is built by ensuring that every prediction comes from an N0 model that has **never seen** that sample during its own training.
- **The Guarantee:** The Meta-Model learns to combine **generalized** and **unbiased** predictions, thereby preserving the integrity of the learning process.

2.4. Stacking and Blending

B. The Concept of Stacking

- **The general principle of Stacking**
- **Step 1: Splitting the training set**
- Split (X_{train}, y_{train}) into K folds (K from 5 to 10, often $K = 5$).

2.4. Stacking and Blending

B. The Concept of Stacking

- **The general principle of Stacking**
- **Step 2: Main loop (generating the meta-features)**
- For each fold $k \in \{1, 2, \dots, K\}$:
  - **Train** the base models on the $K - 1$ folds (all but fold k).
  - **Predict** with the trained models on the validation fold (fold k).
    - These predictions are called *out-of-fold* (OOF) because the model has never seen them during training.
  - **Store** the predictions of fold k.
- After the loop, all the predictions are concatenated to form the new training set of the meta-model:

$$Z_{train} = [\hat{y}_1^{OOF}, \hat{y}_2^{OOF}, \dots, \hat{y}_K^{OOF}]$$

and the associated target remains y_{train}.

2.4. Stacking and Blending

B. The Concept of Stacking

- **The general principle of Stacking**
- **Step 3: Training the meta-model**
- Train the level 1 model (e.g., logistic regression, ridge, GBM) on (Z_{train}, y_{train}).
- The meta-model learns to weight or correct the errors of the base models.

2.4. Stacking and Blending

B. The Concept of Stacking

- **The general principle of Stacking**
- For the test set X_{test}:
  - **Re-train** every base model on the whole X_{train}.
  - **Generate** their predictions on X_{test}.

$$Z_{test} = [\hat{y}_1^{test}, \hat{y}_2^{test}, .,.. \hat{y}_K^{test}]$$

  - **Use** the meta-model (trained on Z_{train}) to produce the final prediction:

$$\hat{y}_{final} = f(Z_{test})$$

2.4. Stacking and Blending

C. The Concept of Blending

- K-**Fold Stacking** is the most robust and best-performing architecture,
  - because it uses **all** the training data (D_{train}) to generate the **meta-features** (out-of-fold) and to train the meta-model.
- However, this method is **complex to implement** and **very computationally expensive** (it requires K trainings of each base model, plus one final re-training).
- There is a simpler and faster alternative: **Blending**.

2.4. Stacking and Blending

C. The Concept of Blending

- **Blending** is a variant of **Stacking**, designed to be **faster and simpler to implement**, while following the same philosophy:

  Combining several base models through a final model (*meta-model*).
- The essential difference is that **blending does not rely on cross-validation (K-Fold)**.
- Instead, it uses **a fixed subset of the training set** (called a *validation set*) to build the *meta-features*.

2.4. Stacking and Blending

C. The Concept of Blending

- **The general principle of Stacking**

1. **Splitting the data**:
   - Training set X_{train}
   - Validation set X_{val}
   - Test set X_{test}
2. **Training the base models**: Each level 0 model is trained only on X_{train} .

2.4. Stacking and Blending

C. The Concept of Blending

- **The general principle of Stacking**

3. **Predictions on the validation set**:

   Each model produces a prediction on X_{val}

$$Z_{val} = [\hat{y}_1^{val}, \hat{y}_2^{val}, \ldots, \hat{y}_K^{val}]$$

4. **Training the meta-model**: The level 1 model is trained on (Z_{val}, y_{val}).

2.4. Stacking and Blending

C. The Concept of Blending

- **The general principle of Stacking**
- **Final prediction:**
- The base models predict on $X_{test} \rightarrow Z_{test}$
- The meta-model produces the final prediction:

$$\hat{y}_{final} = f(Z_{test})$$

2.4. Stacking and Blending

D. Blending vs. Stacking

Strategic Aspect	Blending (Fast Approach)	Stacking (Robust Approach)
Implementation Complexity	Low. It is a simple train_test_split split followed by two fit() calls.	High. It requires a K-Fold cross-validation loop, management of the OOF (Out-of-Fold) indices, and a final re-training.
Data Efficiency	Low (inefficient). This is its main drawback.	High (maximum).
	The L0 models are under-trained (they have seen only ~80% of the data).	The L0 models are trained on 100% of the data (via the K−1 folds + final re-training).
	The Meta-Model is under-trained (it has seen only ~20% of the data).	The Meta-Model is trained on 100% of the OOF predictions.
Risk of Data Leakage	None (if properly implemented). The separation is clean.	Low (if properly implemented).
Robustness	Average. The Meta-Model's performance depends heavily on the random Hold-Out split.	High. Performance is averaged over K folds, making it more stable.
Training Speed	Fast.	Slow (at least K+1 times slower than Blending).

2. Decision Trees and Ensemble Methods
Conclusion

- **Ensemble methods**: Bagging, Boosting, Stacking and Blending share a common philosophy:

  "**Several imperfect models are better than a single perfect one.** "

  - **Bagging** reduces the variance.
  - **Boosting** reduces the bias.
  - **Stacking** learns to combine the best of both worlds.